Customer Segmentation using K-Means Clustering

Problem Statement

In today's competitive market, businesses need to understand their customers to deliver personalized experiences and increase revenue. However, customers differ in terms of age, income, purchasing habits, and loyalty, making it difficult to target them effectively.

This project aims to analyze customer purchasing behavior using features such as age, annual income, purchase amount, purchase frequency, region, and loyalty score. The objective is to group customers into meaningful segments based on their similarities.

By performing customer segmentation, businesses can:

● Identify high-value and low-value customers

● Design targeted marketing strategies

● Improve customer retention and loyalty

● Optimize product recommendations The dataset used is synthetic and created for learning purposes.

Objective

The main objective of this project is to segment customers into different groups based on their purchasing behavior using K-Means Clustering.

Key featuers

We will use these features to understand customer behavior:

- Age
- Annual Income
- Purchase Amount
- Purchase Frequency
- Region
- Loyalty Score

Import Libraries

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Load Data

```
data = pd.read_csv('Customer Purchasing Behaviors.csv')
data
```

	user_id	age	annual_income	purchase_amount	loyalty_score	region	purchase_frequency
0	1	25	45000	200	4.5	North	12
1	2	34	55000	350	7.0	South	18
2	3	45	65000	500	8.0	West	22
3	4	22	30000	150	3.0	East	10
4	5	29	47000	220	4.8	North	13
...	...	...	...	...	...	...	...
233	234	40	60000	450	7.2	West	20
234	235	38	59000	430	6.9	North	20
235	236	54	74000	630	9.4	South	27
236	237	32	52000	360	5.8	West	18
237	238	31	51000	340	5.6	North	17

238 rows × 7 columns

Basic Checks

```
# shape : gives number of rows and columns
data.shape
```

Show hidden output

```
# head: print first 5 rows
data.head()
```

Show hidden output

```
# tail : print last 5 rows
data.tail()
```

Show hidden output

```
# info: memory usage
data.info()
```

Show hidden output

Provides information about data types, non-null values, and memory usage.

```
# print all columns
data.columns
```

```
Index(['user_id', 'age', 'annual_income', 'purchase_amount', 'loyalty_score',
       'region', 'purchase_frequency'],
      dtype='object')
```

```
# check the data type
data.dtypes
```

Show hidden output

```
# statiscal report for numerical column
data.describe()
```

Show hidden output

Data Cleaning

Check for Misisng values

```
data.isnull().sum()
```

Show hidden output

Check for duplicates

```
data.duplicated().sum()
```

```
np.int64(0)
```

Exploratory Data Analysis

Univariate Analysis

```
#Numerical features distribution

# Distribution of Age
plt.figure(figsize=(5,2))
sns.histplot(data, x="age", kde=True)
plt.title("Distribution of Age")
plt.show()
print("\n")

# Distribution of Annual Income
plt.figure(figsize=(5,2))
sns.histplot(data, x="annual_income", kde=True)
plt.title("Distribution of Annual Income")
plt.show()
print("\n")

# Distribution of Purchase Amount
plt.figure(figsize=(5,2))
sns.histplot(data, x="purchase_amount", kde=True)
plt.title("Distribution of Purchase Amount")
plt.show()
```

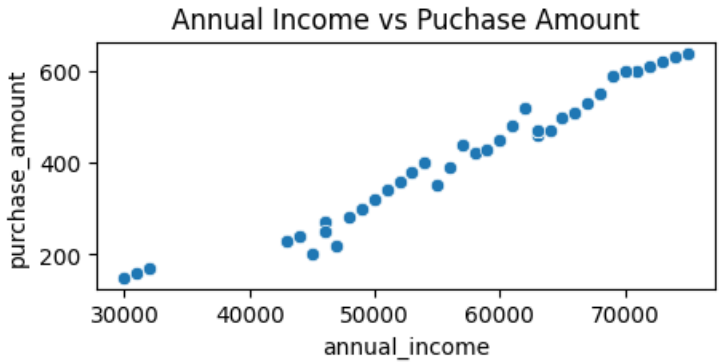
Show hidden output

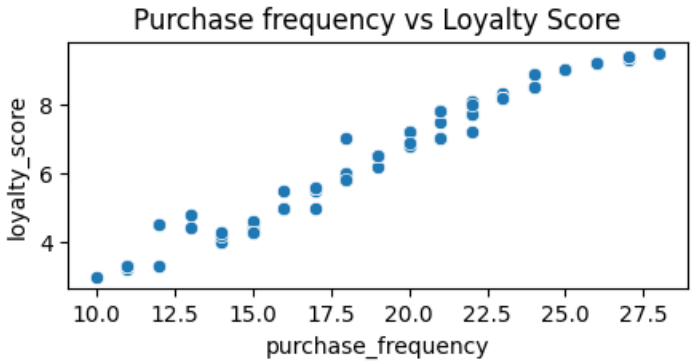
Bivariate

```
# Income vs Purchase
plt.figure(figsize=(5,2))
sns.scatterplot(data=data, x="annual_income", y="purchase_amount")
plt.title("Annual Income vs Puchase Amount")
```

```
plt.show()
print("\n")

# Purchase frequency vs Loyalty score
plt.figure(figsize=(5,2))
sns.scatterplot(data=data, x="purchase_frequency", y="loyalty_score")
plt.title("Purchase frequency vs Loyalty Score")
plt.show()
```





Multivariate

Pairplot

```
sns.pairplot(data, hue="region")
```

Show hidden output

```
# finding corr
num_col = data.select_dtypes(include=["int64","float64"])
num_col.corr()
```

Show hidden output

```
sns.heatmap(num_col.corr(),annot=True)
```

Show hidden output

Patterns and Trends

- Higher income → higher purchase amount
- Higher purchase frequency → higher loyalty
- Customers show different buying behaviors

- Clear groups like high-value and low-value customers

One Hot Encoding

```
data = pd.get_dummies(data, columns=['region'], drop_first=True)
```

```
data = data.astype(int)
```

```
data
```

	user_id	age	annual_income	purchase_amount	loyalty_score	purchase_frequency	region_North	regi
0	1	25	45000	200	4	12	1	
1	2	34	55000	350	7	18	0	
2	3	45	65000	500	8	22	0	
3	4	22	30000	150	3	10	0	
4	5	29	47000	220	4	13	1	
...	...	...	...	...	...	...	...	
233	234	40	60000	450	7	20	0	
234	235	38	59000	430	6	20	1	
235	236	54	74000	630	9	27	0	
236	237	32	52000	360	5	18	0	
237	238	31	51000	340	5	17	1	

238 rows × 9 columns

```
data = data.drop(columns=['user_id'])
```

Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

features = ['age', 'annual_income', 'purchase_amount',
            'loyalty_score', 'purchase_frequency',
            'region_North', 'region_South', 'region_West']

scaled_data = scaler.fit_transform(data[features])
```

Feature Selection

```
selected_features = ['annual_income', 'purchase_amount',
                     'purchase_frequency', 'loyalty_score']

scaled_df = pd.DataFrame(scaled_data, columns=features)

X = scaled_df[selected_features]
```

K means

```
from sklearn.cluster import KMeans
model = KMeans(n_clusters=5)
model.fit(X)
```

```
KMeans
KMeans(n_clusters=5)
```

```
df = data.copy()

df['clusters'] = model.predict(X)

df['clusters'].unique()
```

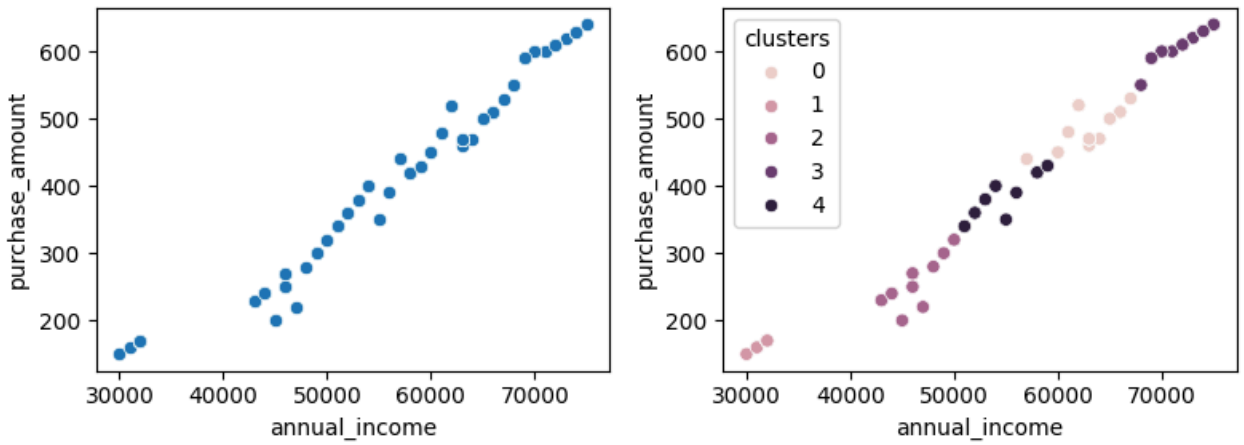
```
array([2, 4, 0, 1, 3], dtype=int32)
```

Visualise clusters

```
#income vs purchase amount
fig, ax = plt.subplots(1,2,figsize=(8,3))

sns.scatterplot(data=df, x='annual_income', y='purchase_amount', ax=ax[0])
sns.scatterplot(data=df, x='annual_income', y='purchase_amount', hue='clusters', ax=ax[1])

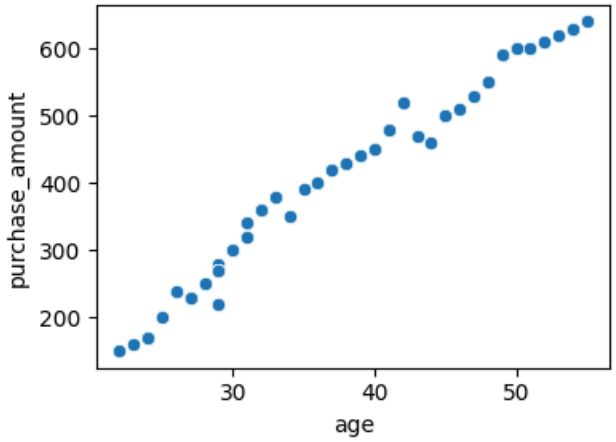
plt.tight_layout()
```

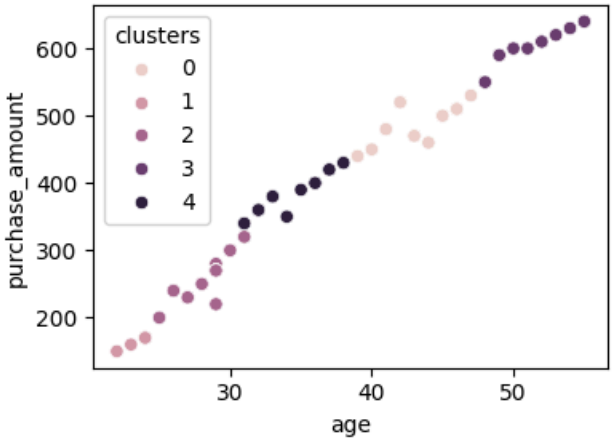



```
#age vs purchase amount
fig, ax = plt.subplots(1,2,figsize=(8,3))

sns.scatterplot(data=df, x='age', y='purchase_amount', ax=ax[0])
sns.scatterplot(data=df, x='age', y='purchase_amount', hue='clusters', ax=ax[1])

plt.tight_layout()
```

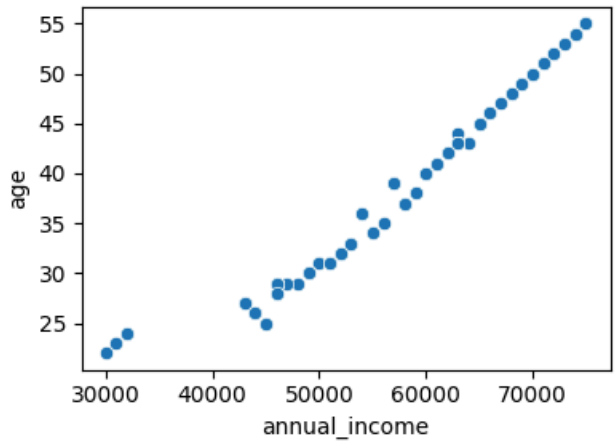


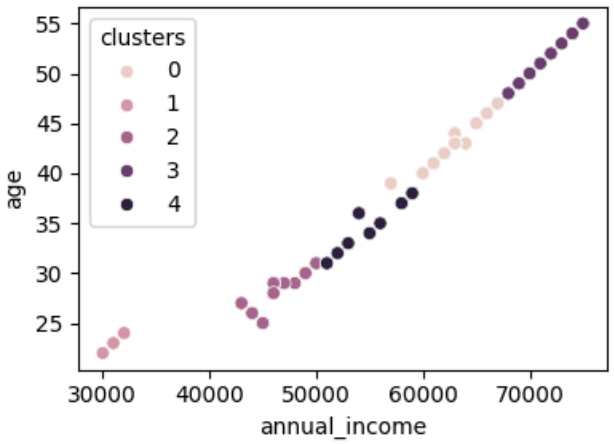



```
#income vs age
fig, ax = plt.subplots(1,2,figsize=(8,3))

sns.scatterplot(data=df, x='annual_income', y='age', ax=ax[0])
sns.scatterplot(data=df, x='annual_income', y='age', hue='clusters', ax=ax[1])

plt.tight_layout()
```






```
# Labels

labels = model.predict(X)
labels
```

```
array([2, 4, 0, 1, 2, 0, 4, 2, 3, 4, 2, 0, 4, 4, 3, 1, 0, 2, 0, 2, 3, 1,
       0, 4, 4, 0, 4, 2, 3, 0, 4, 0, 2, 2, 3, 2, 3, 2, 3, 2, 0, 4, 0, 4,
       3, 2, 0, 3, 4, 1, 3, 2, 0, 0, 4, 3, 4, 4, 0, 3, 2, 4, 3, 2, 0, 0,
       3, 1, 3, 2, 0, 4, 0, 4, 3, 2, 0, 3, 4, 1, 3, 2, 0, 0, 4, 3, 4, 4,
       0, 3, 2, 4, 3, 2, 0, 0, 3, 1, 3, 2, 0, 4, 0, 4, 3, 2, 0, 3, 4, 1,
       3, 2, 0, 0, 4, 3, 4, 4, 0, 3, 2, 4, 3, 2, 0, 0, 3, 1, 3, 2, 0, 4,
       0, 4, 3, 2, 0, 3, 4, 1, 3, 2, 0, 0, 4, 3, 4, 4, 0, 3, 2, 4, 3, 2,
       0, 0, 3, 1, 3, 2, 0, 4, 0, 4, 3, 2, 0, 3, 4, 1, 3, 2, 0, 0, 4, 3,
       4, 4, 0, 3, 2, 4, 3, 2, 0, 0, 3, 1, 3, 2, 0, 4, 0, 4, 3, 2, 0, 3,
       4, 1, 3, 2, 0, 0, 4, 3, 4, 4, 0, 3, 2, 4, 3, 2, 0, 0, 3, 1, 3, 2,
       0, 4, 0, 4, 3, 2, 0, 3, 4, 1, 3, 2, 0, 0, 4, 3, 4, 4], dtype=int32)
```

```
from sklearn.metrics import silhouette_score
silhouette_score(X,labels)
```

```
np.float64(0.6018514526444354)
```

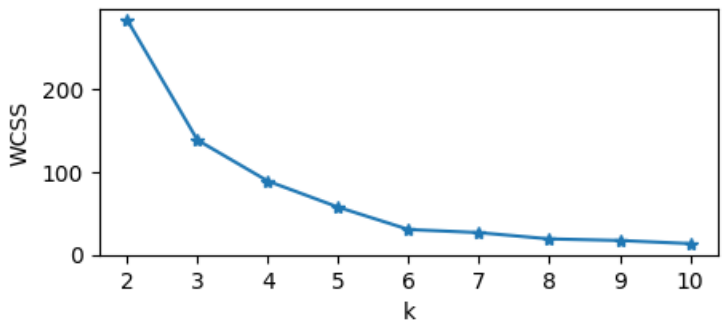
```
wcss = []

for k in range(2,11):
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(X)
    wcss.append(model.inertia_)
```

```
wcss
```

```
[284.52456286698816,
 138.71873305240638,
 89.02554316692563,
 57.3936296250455,
 30.331032132076388,
 26.55510582140709,
 18.935180227701558,
 17.028663931213337,
 13.251802262091015]
```

```
plt.figure(figsize=(5,2))
plt.plot(range(2,11), wcss, marker='*')
plt.xlabel('k')
plt.ylabel('WCSS')
plt.show()
```




```
df1 = data.copy()

model = KMeans(n_clusters=4)
model.fit(X)
```

```
KMeans
KMeans(n_clusters=4)
```

```
df1['clusters'] = model.predict(X)
df1['clusters'].unique()

labels = model.predict(X)

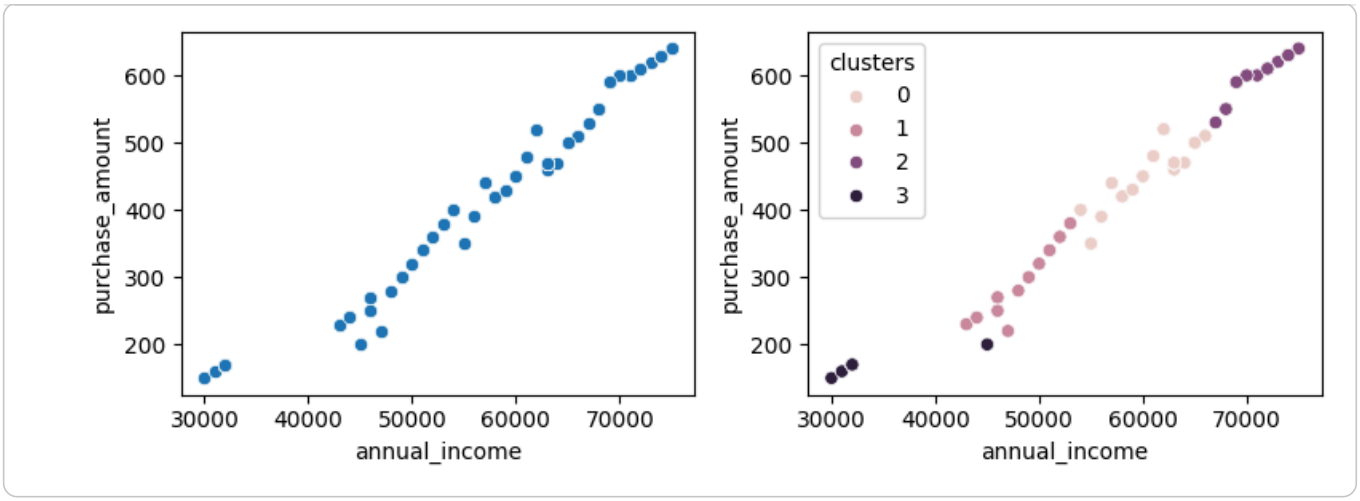
silhouette_score(X, labels)
```

```
np.float64(0.5364679609297037)
```

```
fig, ax = plt.subplots(1,2,figsize=(8,3))

sns.scatterplot(data=df1, x='annual_income', y='purchase_amount', ax=ax[0])
sns.scatterplot(data=df1, x='annual_income', y='purchase_amount', hue='clusters', ax=ax[1])

plt.tight_layout()
```


purchase_amount
annual_income
clusters
0
1
2
3